

Transformers and LLMs

NEBIUS **ACADEMY**

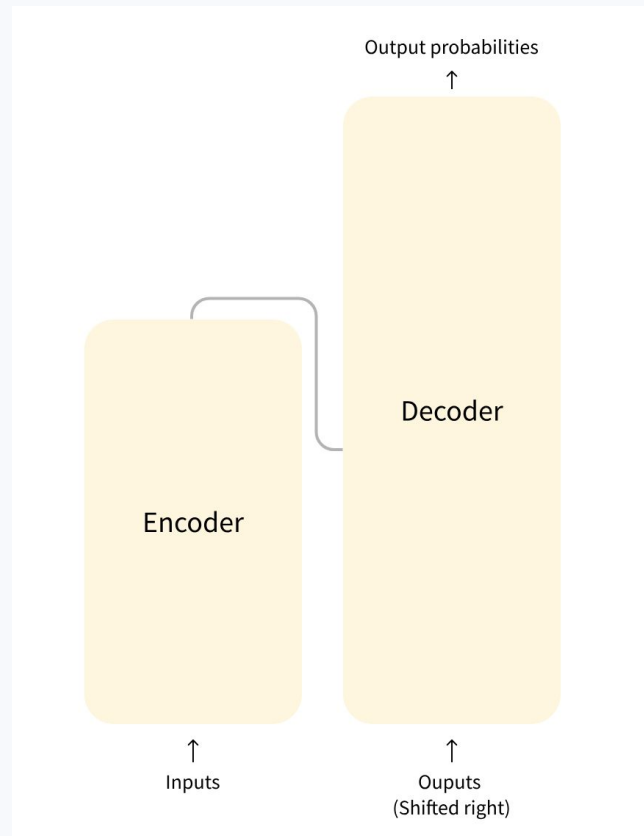
Agenda

- Transformers
- LLMs with transformers
- LLMs at inference time:
 - Sampling
 - top p
 - top k
 - temperature

Transformers

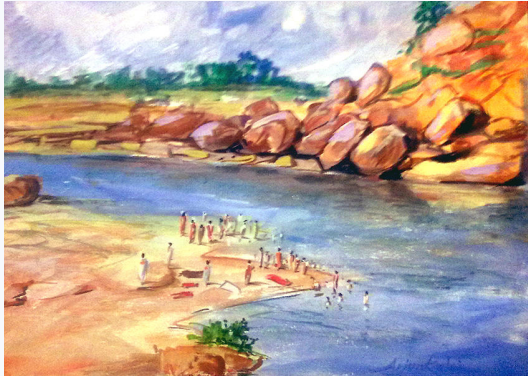
The Transformer is primarily composed of two blocks

- Encoder (left): The encoder receives an input and builds a representation of it (its features).
- Decoder (right): The decoder uses the encoder's representation (features) along with other inputs to generate a target sequence.



Problem with Word Embeddings

They are **static**



The chicken didn't cross the road because **it** was too tired

Solution

- We replace word embeddings with contextualized word vectors.

$$Emb = f(word, context)$$

Solution

Build up the contextual embedding from a token by **selectively** integrating information from all the neighboring words

How?

Solution

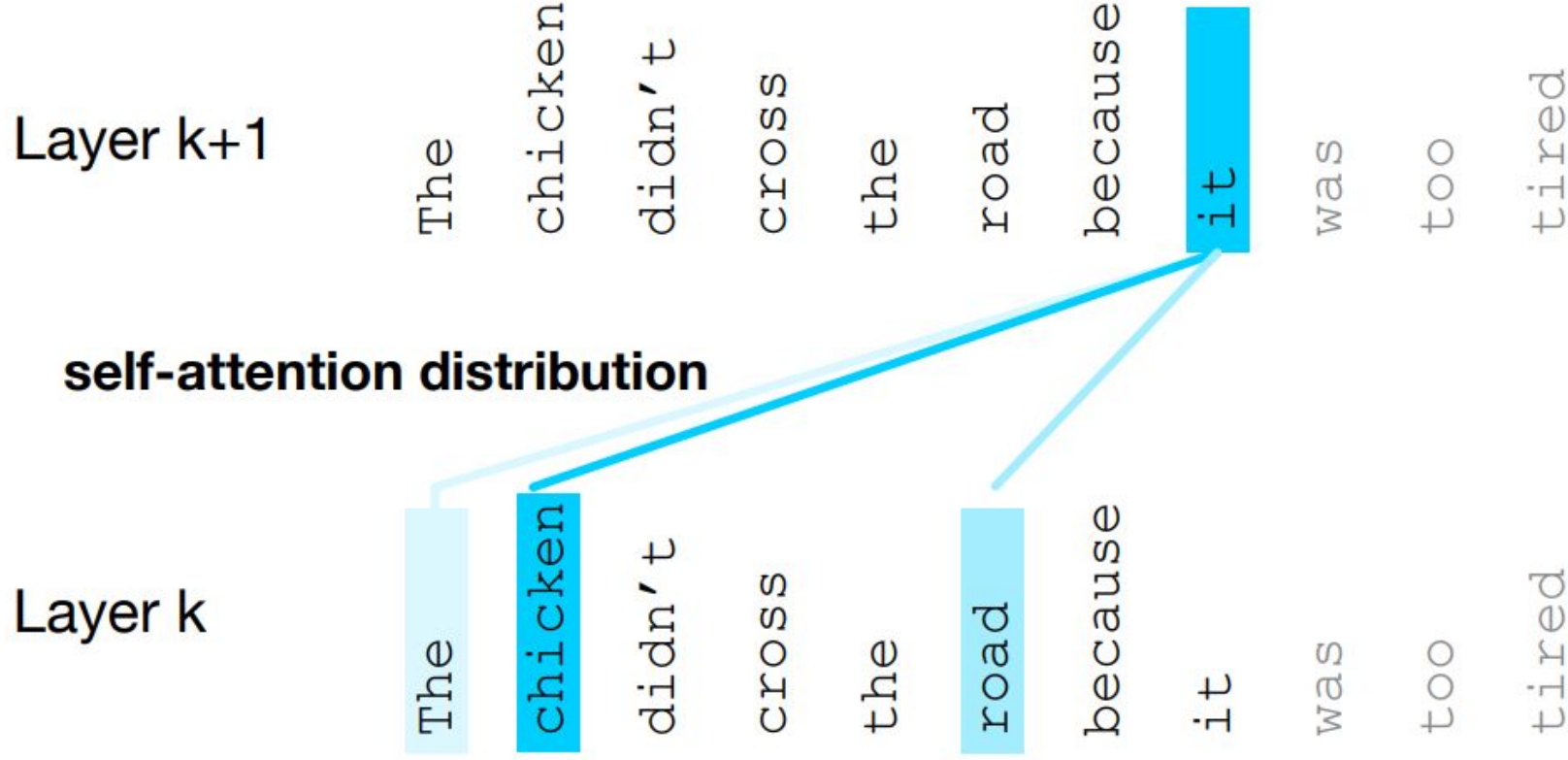
Build up the contextual embedding from a token by **selectively** integrating information from all the neighboring words

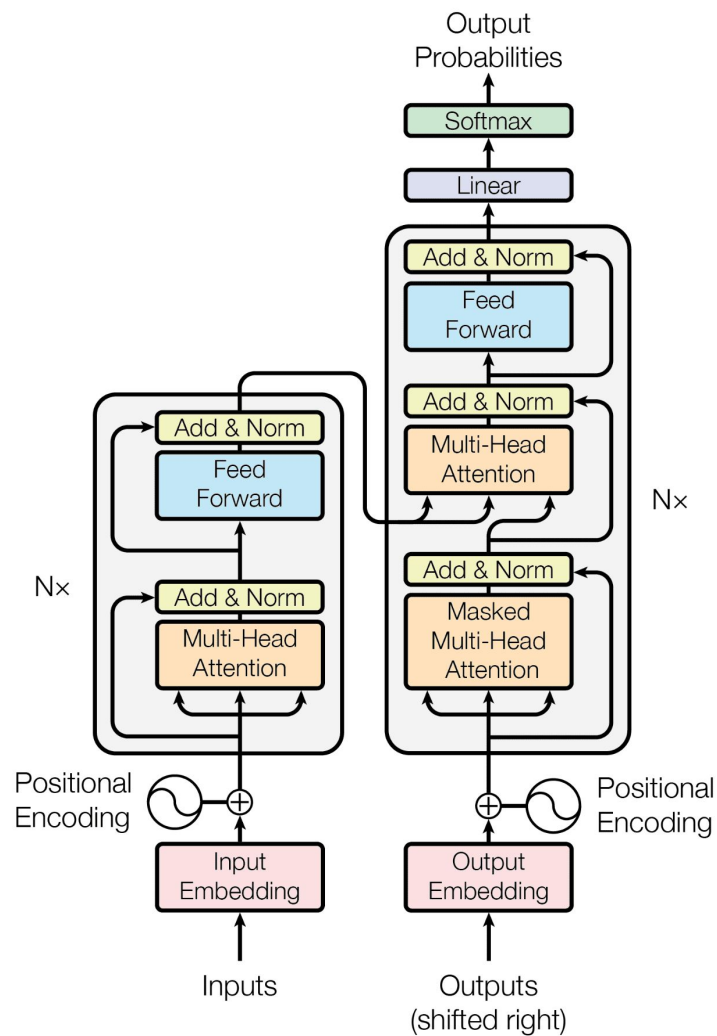
How? **attention!**

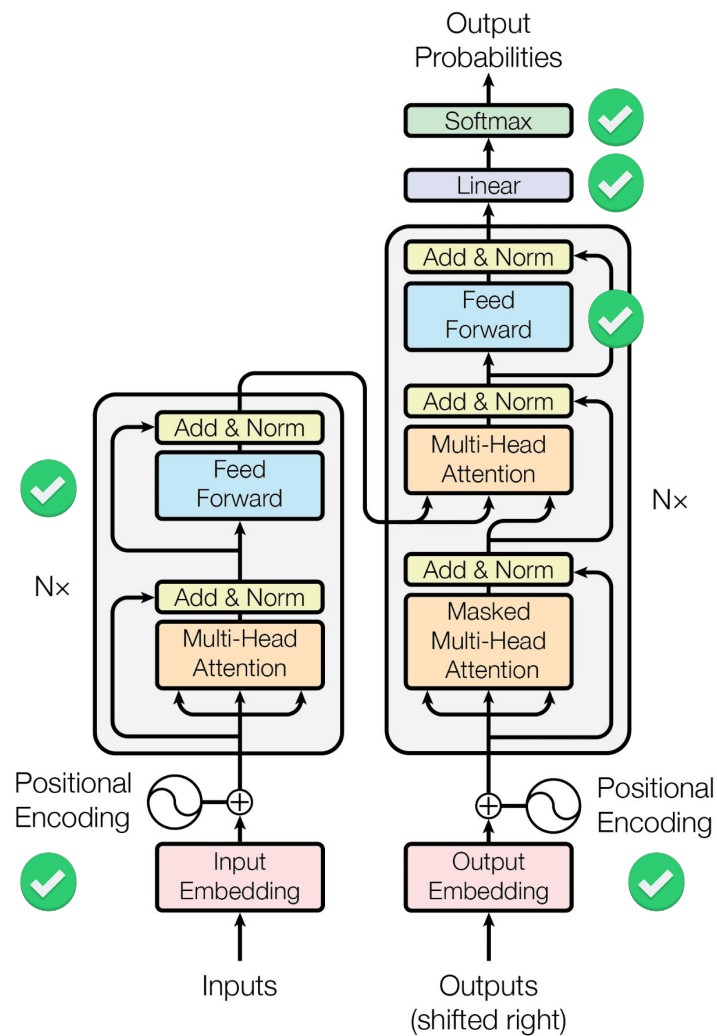
The chicken didn't cross the road because **it** was too tired

The chicken didn't cross the road because **it** was too wide

columns corresponding to input tokens







Attention Is All You Need

Ashish Vaswani*

Google Brain

avaswani@google.com

Noam Shazeer*

Google Brain

noam@google.com

Niki Parmar*

Google Research

nikip@google.com

Jakob Uszkoreit*

Google Research

usz@google.com

Llion Jones*

Google Research

llion@google.com

Aidan N. Gomez* †

University of Toronto

aidan@cs.toronto.edu

Łukasz Kaiser*

Google Brain

lukaszkaizer@google.com

Illia Polosukhin* ‡

illia.polosukhin@gmail.com

Transformer - Today's New Concepts

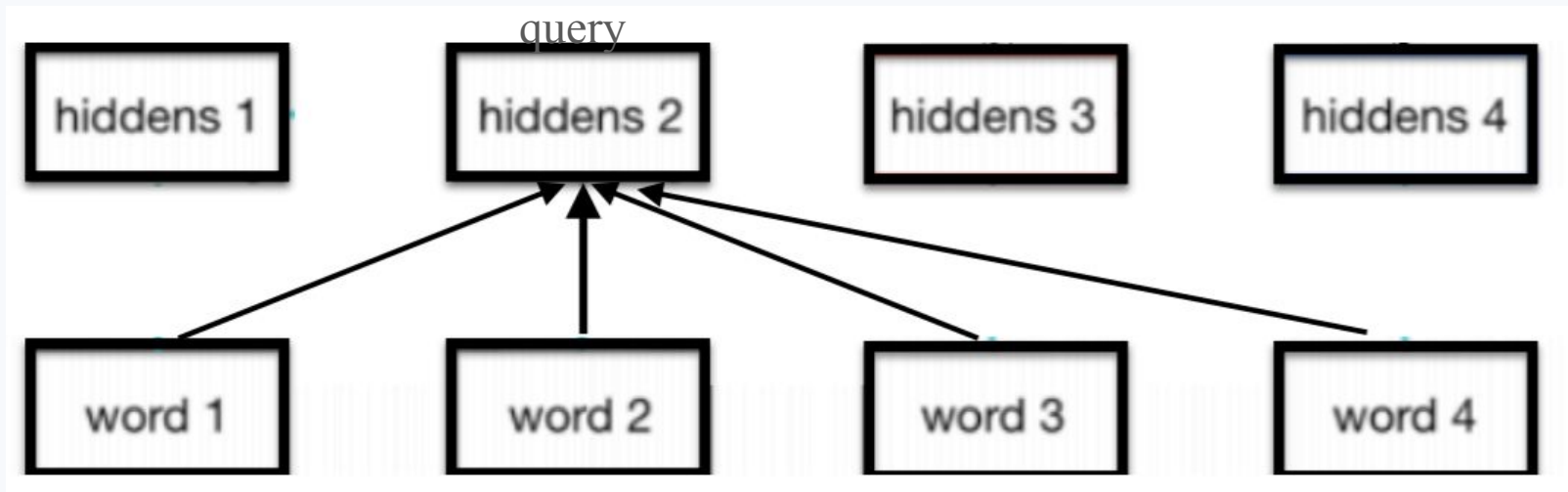
- Self-attention - each token attends to all tokens in previous layer
- Multi-head attention - several attention patterns
- Skip connections
- Layer Norm
- Dropout
- Positional Information

Attention Recap

- **query**: As the current element being compared to the preceding inputs.
- **key**: as a preceding input that is being compared to the current element to determine a similarity
- **value**: a value of a preceding element that gets weighted and summed

Self Attention

- Each element in the sentence attends to **all** other elements → context sensitive encodings!



key 1
val 1

key 2
val 2

key 3
val 3

key 4
val 4

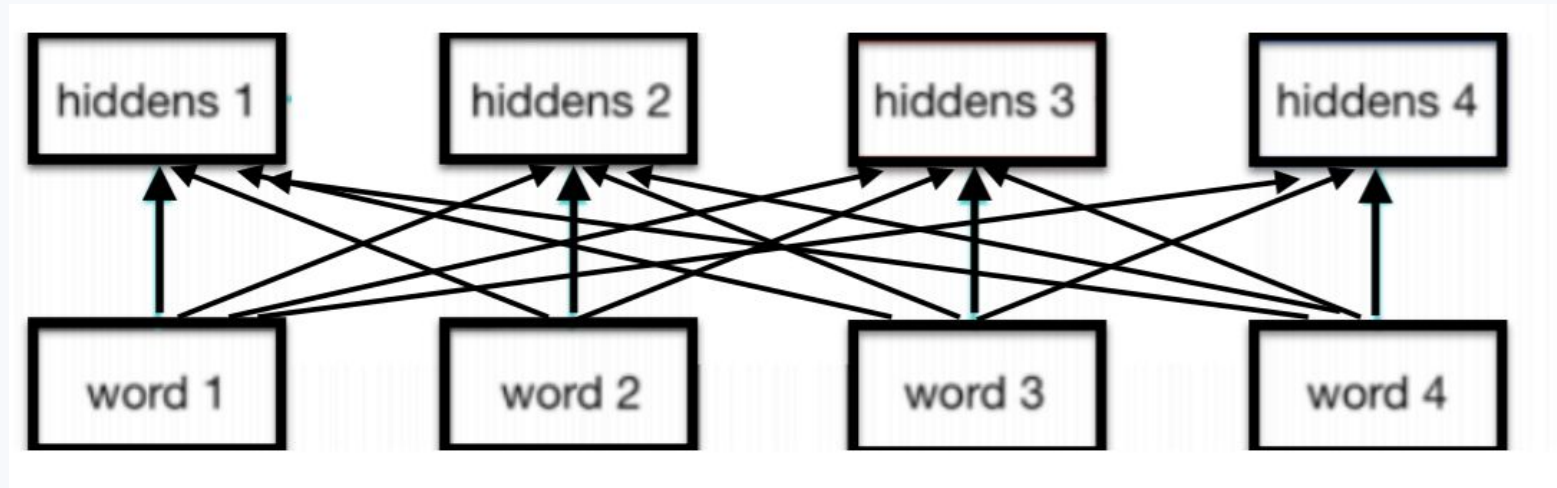
An Actual Attention Head: slightly more complicated

We'll use matrices to project each vector x_i into a representation of its role as query, key, value:

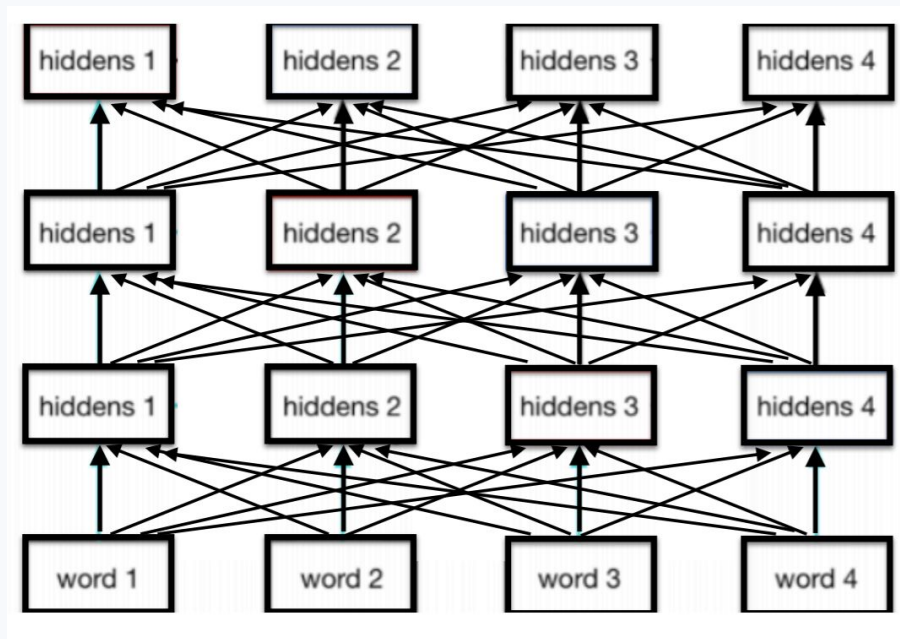
$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

the embedding ($\mathbf{x}$) are also learned as a part of the full training!

Self Attention

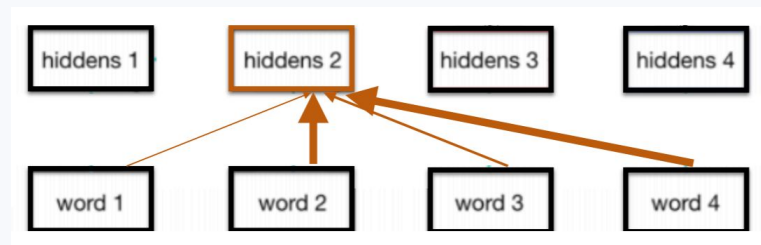
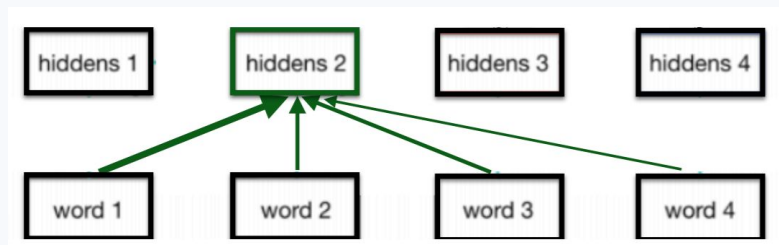


Self Attention



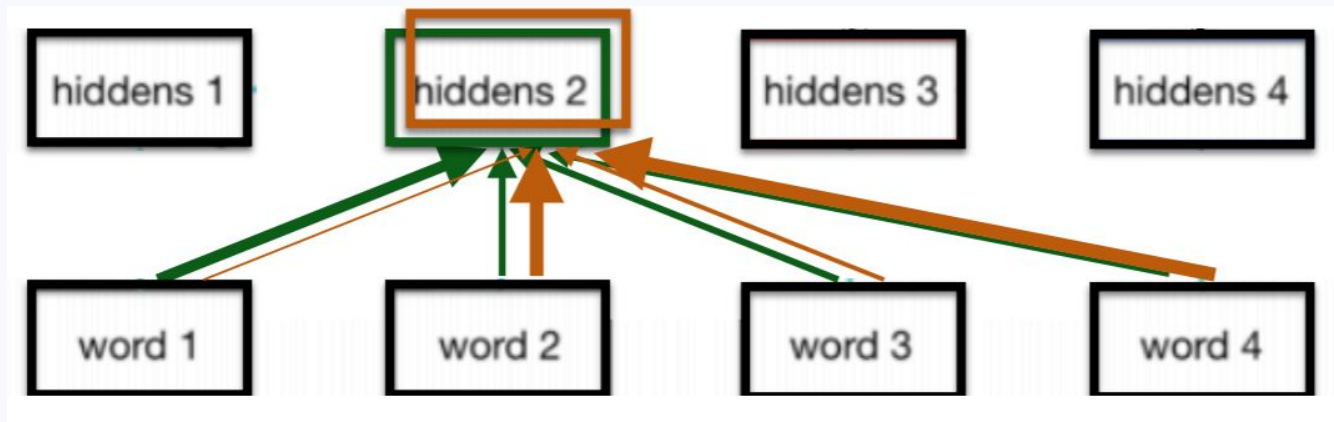
Multi-head Attention

- Two attention patterns



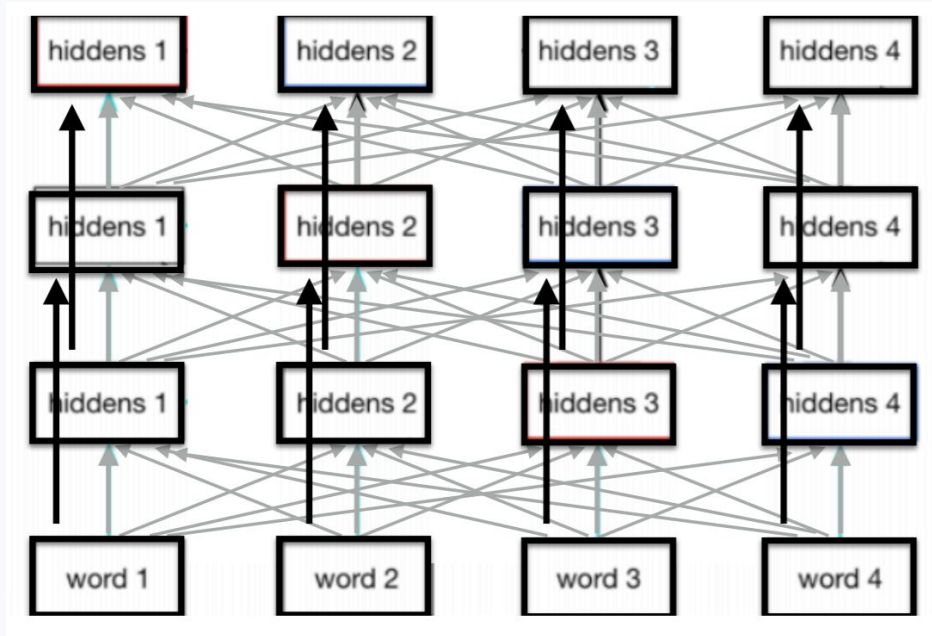
Multi-head Attention

- Why not both?



- Why not **several**?

Skip Connection



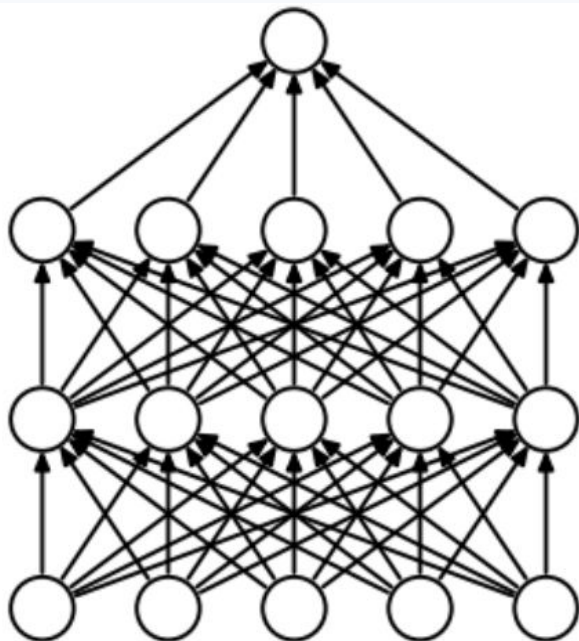
Layer Normaliztion

Layer norm is a variation of the z-score from statistics, applied to a single vector in a hidden layer

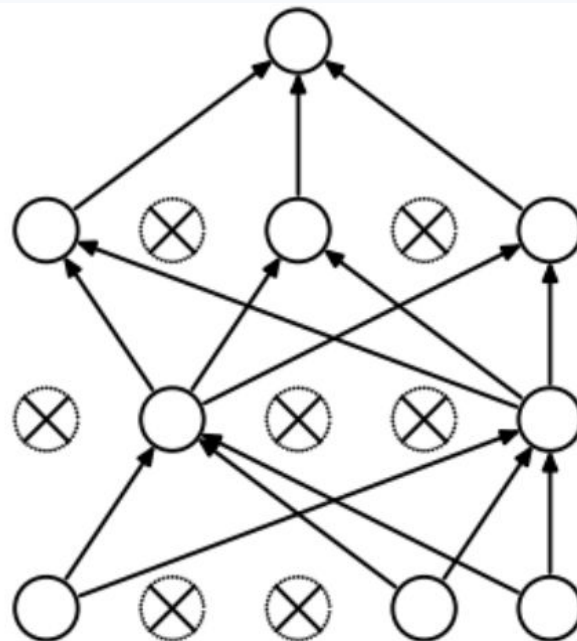
Helps stabalize training and inference

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

Dropout Regularization

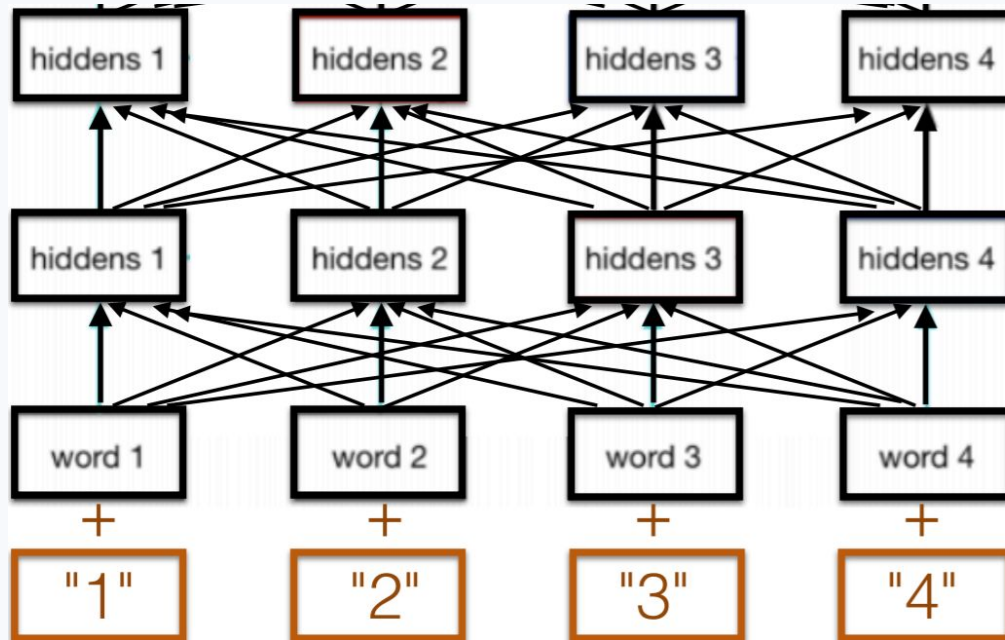


(a) Standard Neural Net



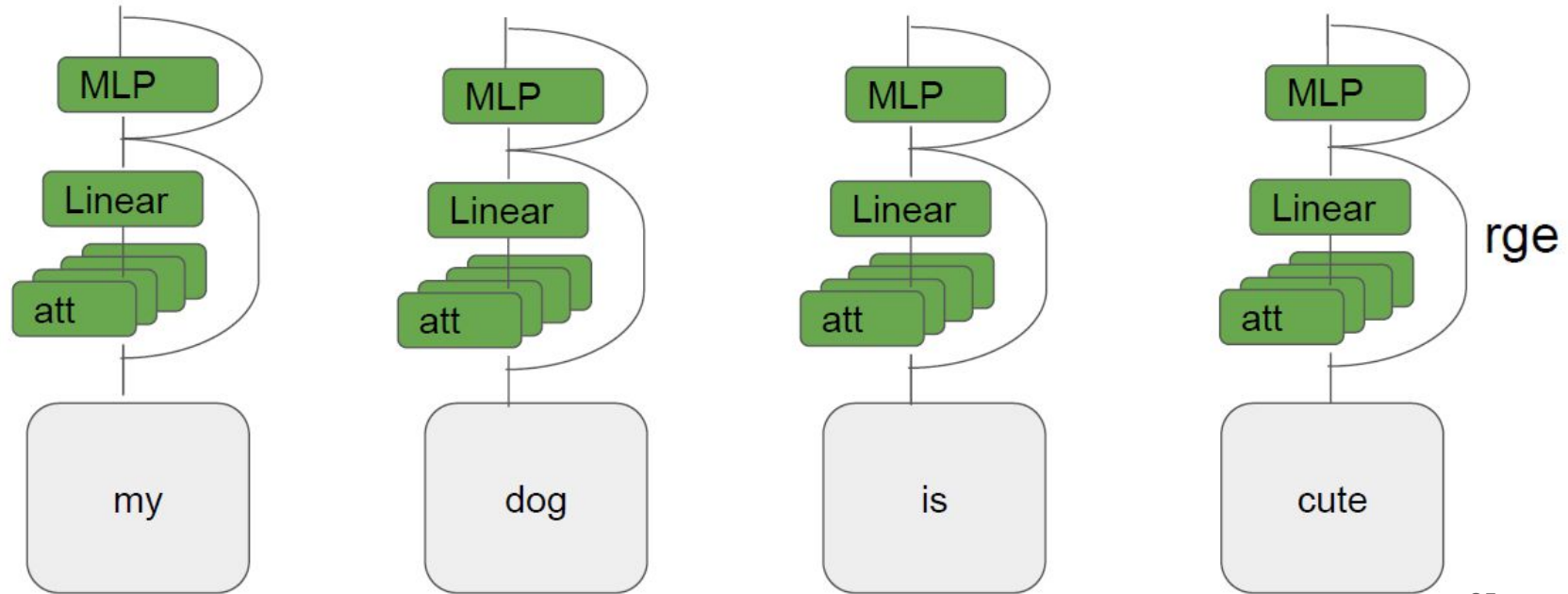
(b) After applying dropout.

Positional Embeddings



Transformer - Put it All Together

-Skip connections, layer normalization, dropout

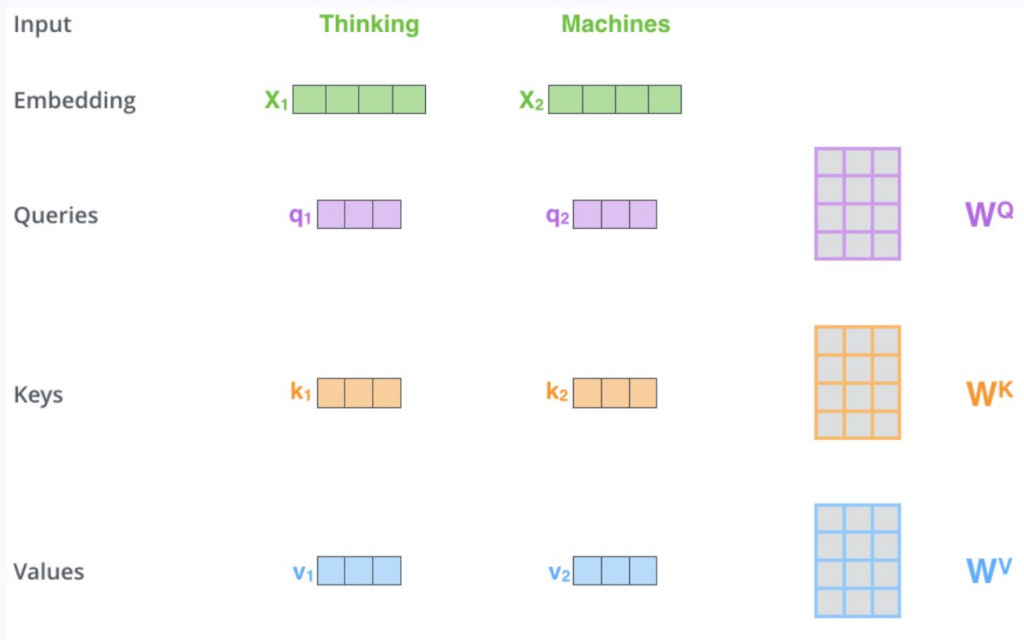


Transformer - Put it All Together

$$\begin{aligned}\mathbf{t}_i^1 &= \text{LayerNorm}(\mathbf{x}_i) \\ \mathbf{t}_i^2 &= \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{x}_1^1, \dots, \mathbf{x}_N^1]) \\ \mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \\ \mathbf{t}_i^4 &= \text{LayerNorm}(\mathbf{t}_i^3) \\ \mathbf{t}_i^5 &= \text{FFN}(\mathbf{t}_i^4) \\ \mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3\end{aligned}$$

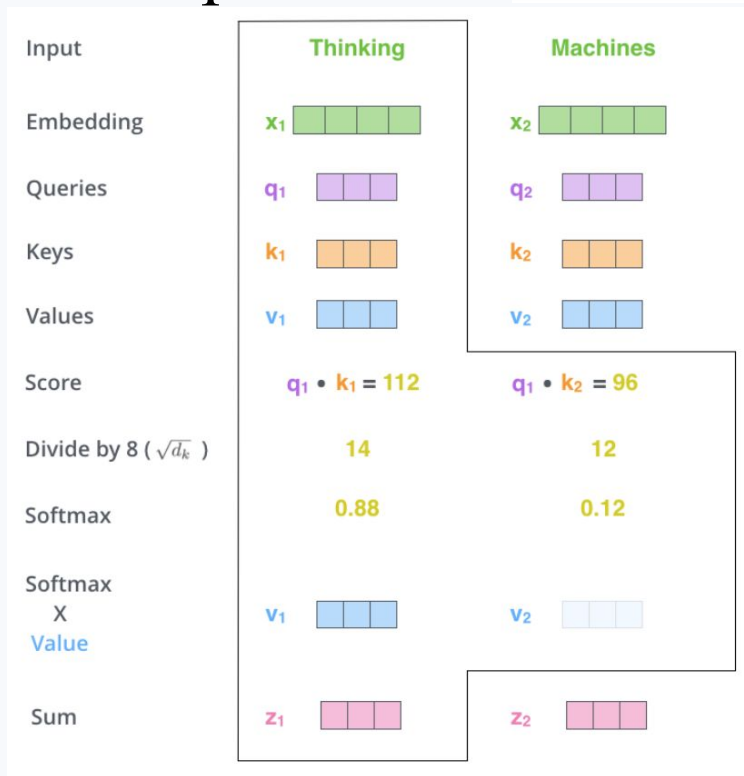
Transformer - Example

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Transformer - Example

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Parallelizing Attention Computation

- To calculate the attention heads we can pack the N tokens of the input sequence into a single matrix X of size $[N \times d]$.
- Each row of X is the embedding of one token of the input.

QKT

N

$q1 \cdot k1$	$q1 \cdot k2$	$q1 \cdot k3$	$q1 \cdot k4$
$q2 \cdot k1$	$q2 \cdot k2$	$q2 \cdot k3$	$q2 \cdot k4$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$q3 \cdot k4$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

What about the decoding? We can Mask the Future!

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Add $-\infty$ to cells in upper triangle
The softmax will turn it to 0

N

q1•k1	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4

Transformer Summary

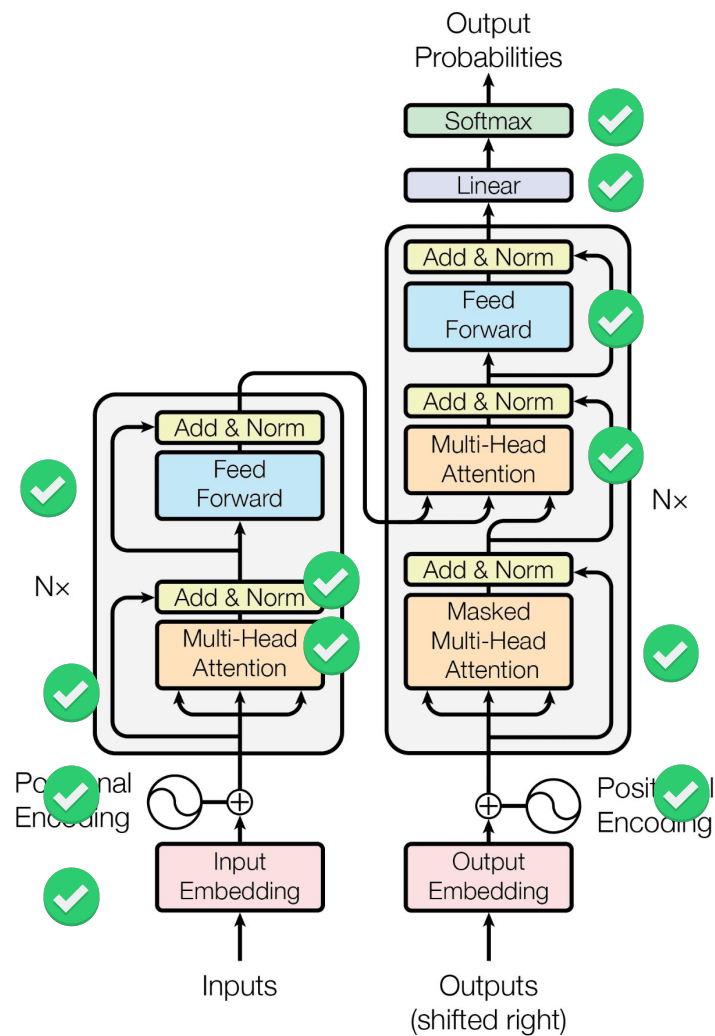
- An alternative to RNN

- **More**

More computation.

More parallelism.

Shorter connections between data points.



Quiz



LLMs

Putting it all together

Remember our simple n-gram model?

Large language models are the same:

- They assign probabilities to sequences of word
- Generate text by sampling possible next words

but different:

- Are trained by learning to guess the next word / hidden word

Simple, and yet we can do so much with them!

Sentiment Analysis

“I like Jackie Chan”

1. We give the language model this string:

The sentiment of the sentence "I like Jackie Chan" is:

2. And see what word it thinks comes next:

$P(\text{positive} | \text{The sentiment of the sentence "I like Jackie Chan" is:})$

$P(\text{negative} | \text{The sentiment of the sentence "I like Jackie Chan" is:})$

Q&A

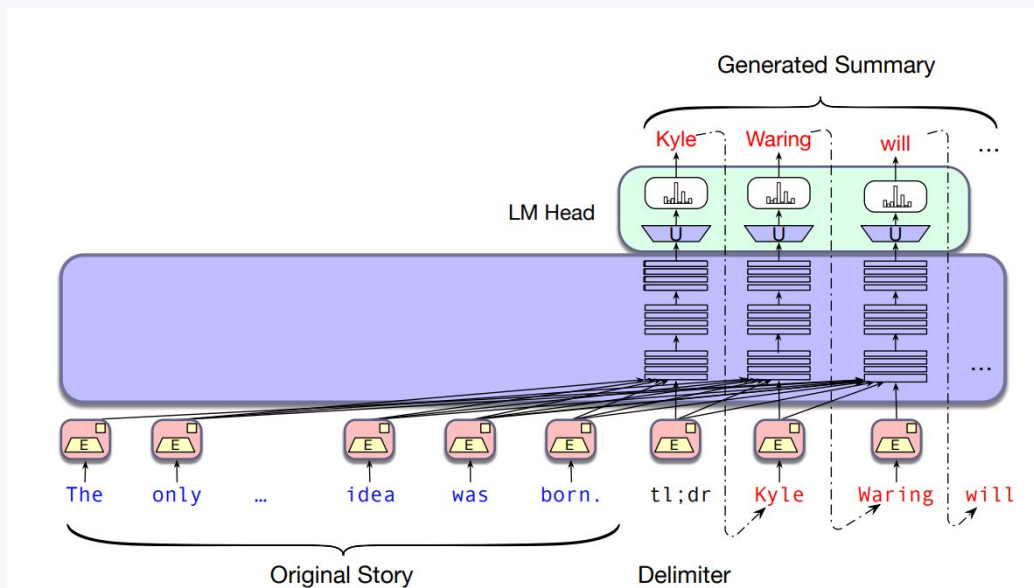
Q: Who wrote the book “The Origin of Species”?

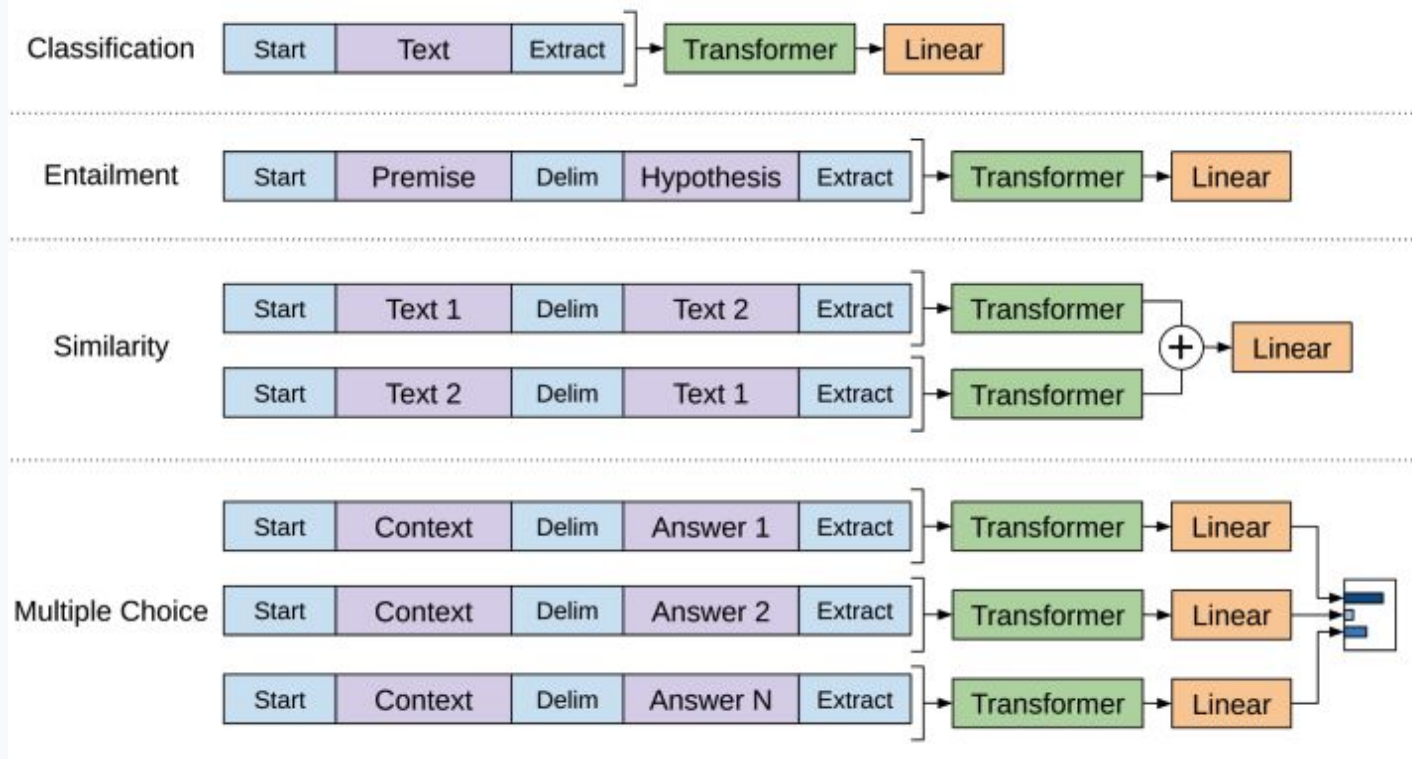
A: If we ask a language model to compute

$P(w|Q: \text{Who wrote the book “The Origin of Species”? A:})$ and look at which words w have high probabilities, we might expect to see Charles.

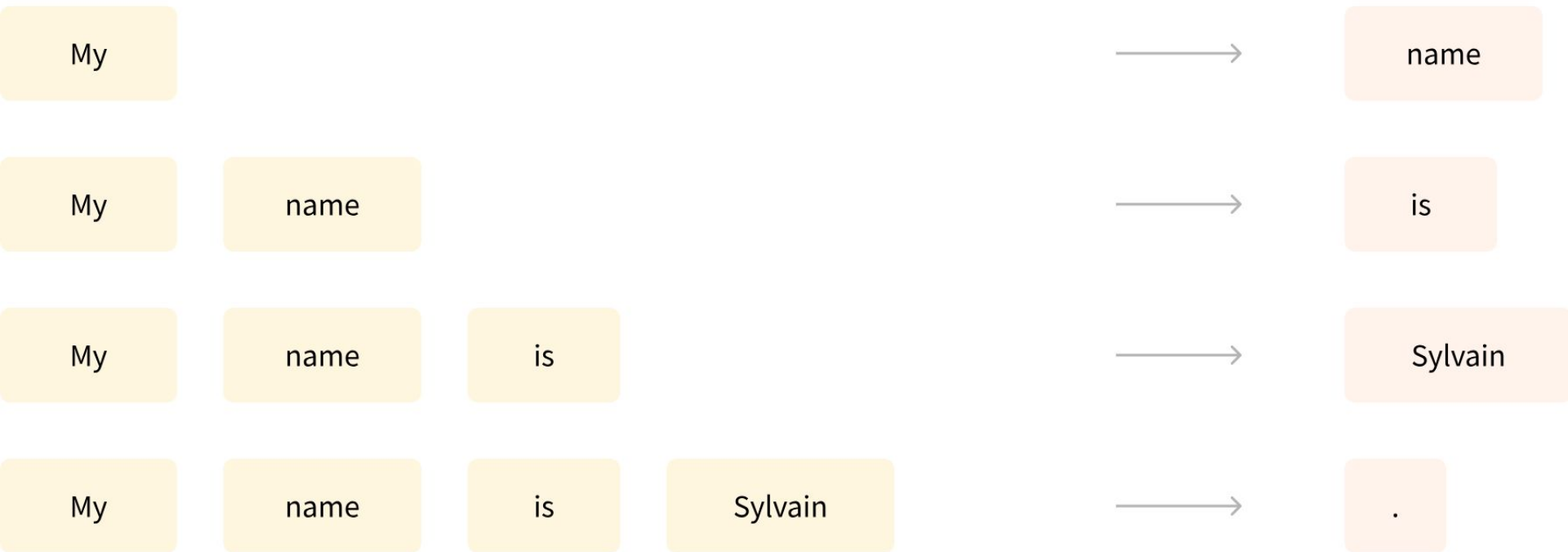
Summarization

LLMs for summarization (example using tl;dr)

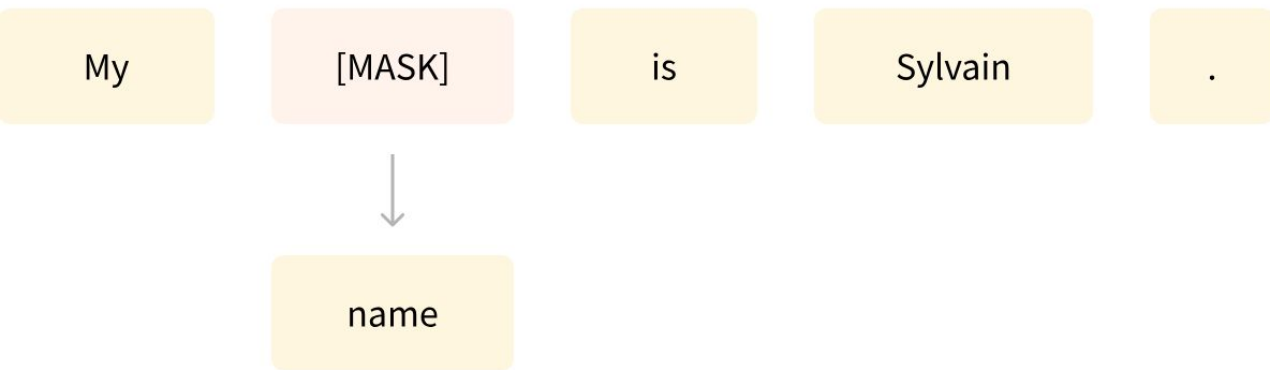




Causal Language Models



Masked Language Models

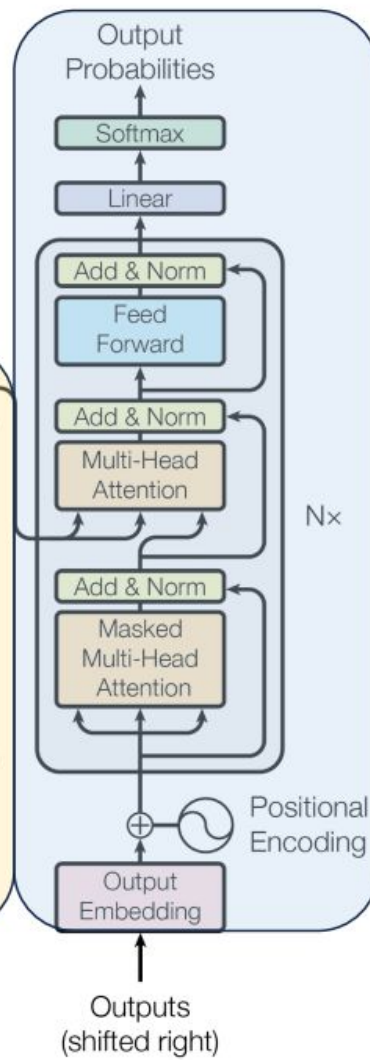
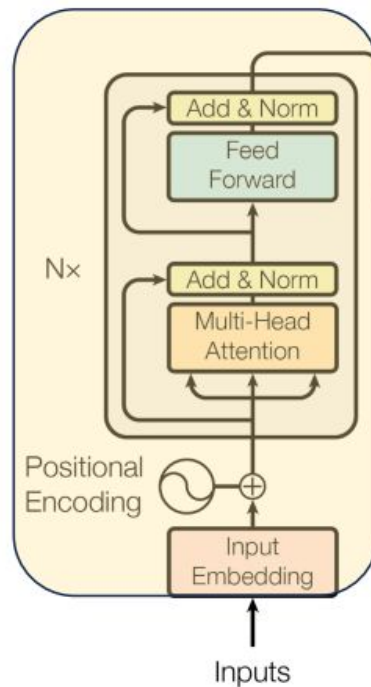


Three Types of Transformer Based LLMs

- **Encoder-only models:** Good for tasks that require understanding of the input, such as sentence classification and named entity recognition.
- **Decoder-only models:** Good for generative tasks such as text generation.
- **Encoder-decoder models** or sequence-to-sequence models: Good for generative tasks that require an input, such as translation or summarization.

BERT
Oct 2018

Representation



GPT
Jun 2018

Generation

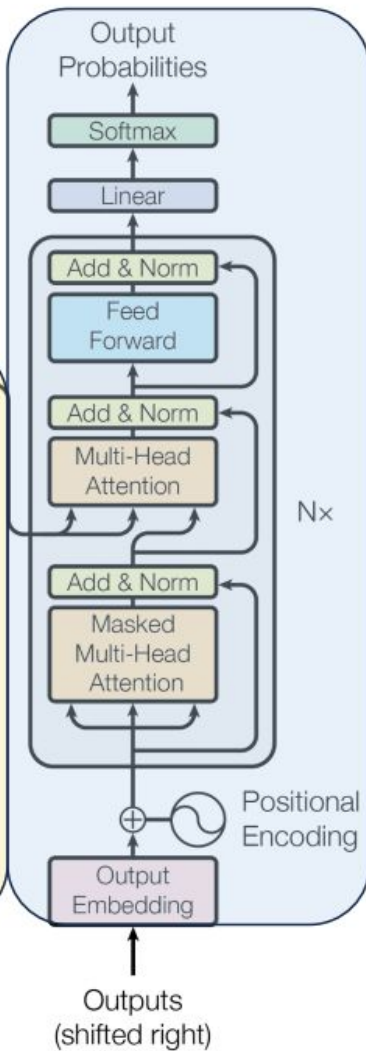
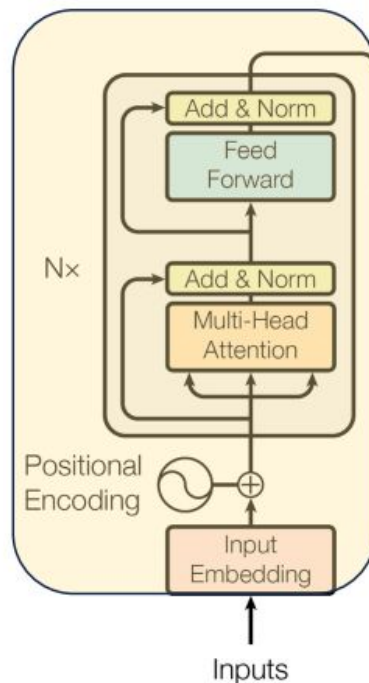
Input – input tokens
Output – hidden states

Model can see all timesteps

Does not usually output tokens,
so no inherent auto-regressivity

Can also be adapted to generate
tokens by appending a module
that maps hidden state
dimensionality to vocab size

Representation



Generation

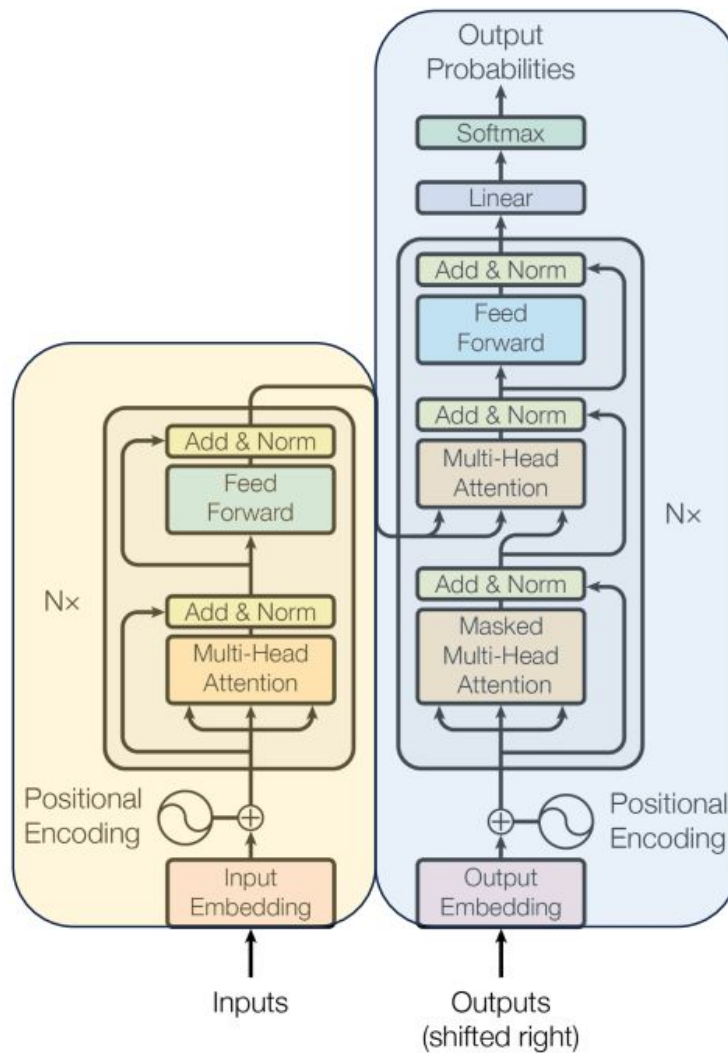
Input – input tokens
Output – hidden states

Model can see all timesteps

Does not usually output tokens,
so no inherent auto-regressivity

Can also be adapted to generate
tokens by appending a module
that maps hidden state
dimensionality to vocab size

Representation



Input – output tokens and
hidden states*
Output – output tokens

Model can only see previous
timesteps

Model is auto-regressive with
previous timesteps' outputs

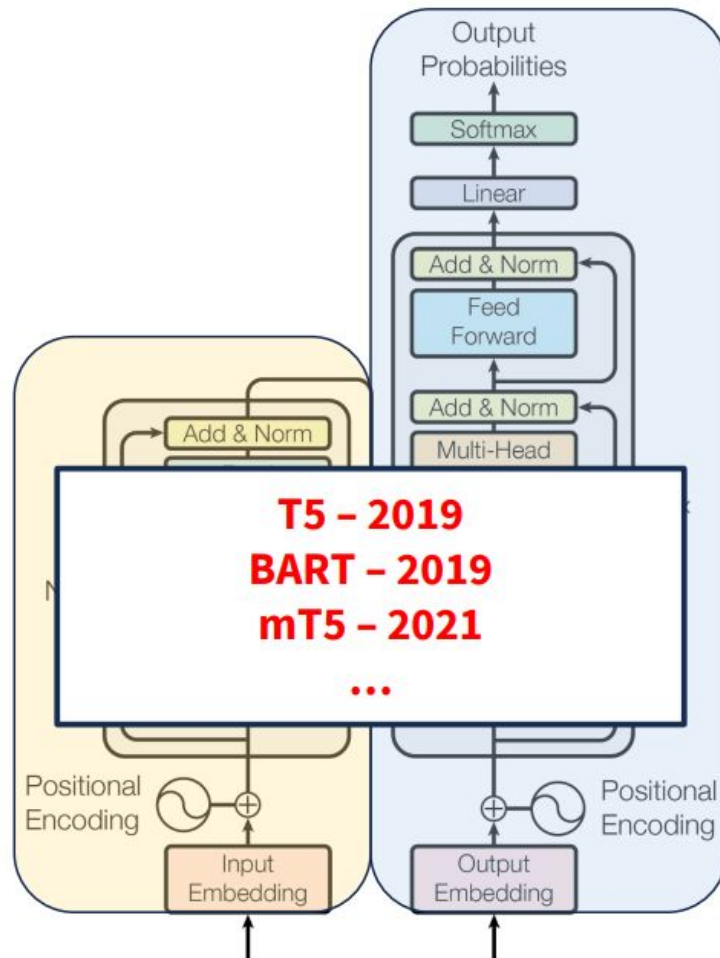
Can also be adapted to
generate hidden states by
looking before token outputs

Generation

BERT – 2018
DistilBERT – 2019
RoBERTa – 2019
ALBERT – 2019
ELECTRA – 2020
DeBERTa – 2020

...

Representation



GPT – 2018
GPT-2 – 2019
GPT-3 – 2020
GPT-Neo – 2021
GPT-3.5 (ChatGPT) – 2022
LLaMA – 2023
GPT-4 – 2023

...

Generation

LLMs

Inference Time

Next week - we'll focus on LLM training

Sampling

This task of choosing a word to generate based on the model's probabilities is called decoding.

The most common method for decoding in LLMs: sampling.

Sampling from a model's distribution over words:

- choose random words according to their probability assigned by the model.
- After each token we'll sample words to generate according to their probability conditioned on our previous choices.
- A transformer language model will give the probability.

```
i ← 1  
wi ∼ p(w)  
while wi ≠ EOS  
    i ← i + 1  
    wi ∼ p(wi | w<i)
```

Sampling

Even though random sampling mostly generate sensible, high-probable words, There are many odd, low- probability words in the tail of the distribution

Each one is low - probability but added up they constitute a large portion of the distribution

Sampling - Quality vs Diversity

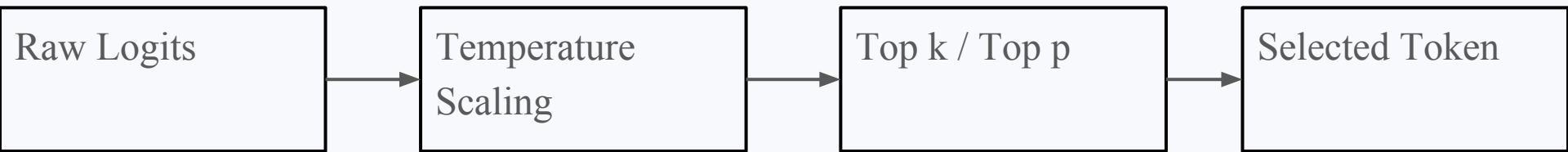
Emphasize high-probability words

- + quality: more accurate, coherent, and factual.
- diversity: boring, repetitive.

Emphasize middle-probability words

- + diversity: more creative, diverse.
- quality: less factual, incoherent

Pipeline



Temperature Sampling

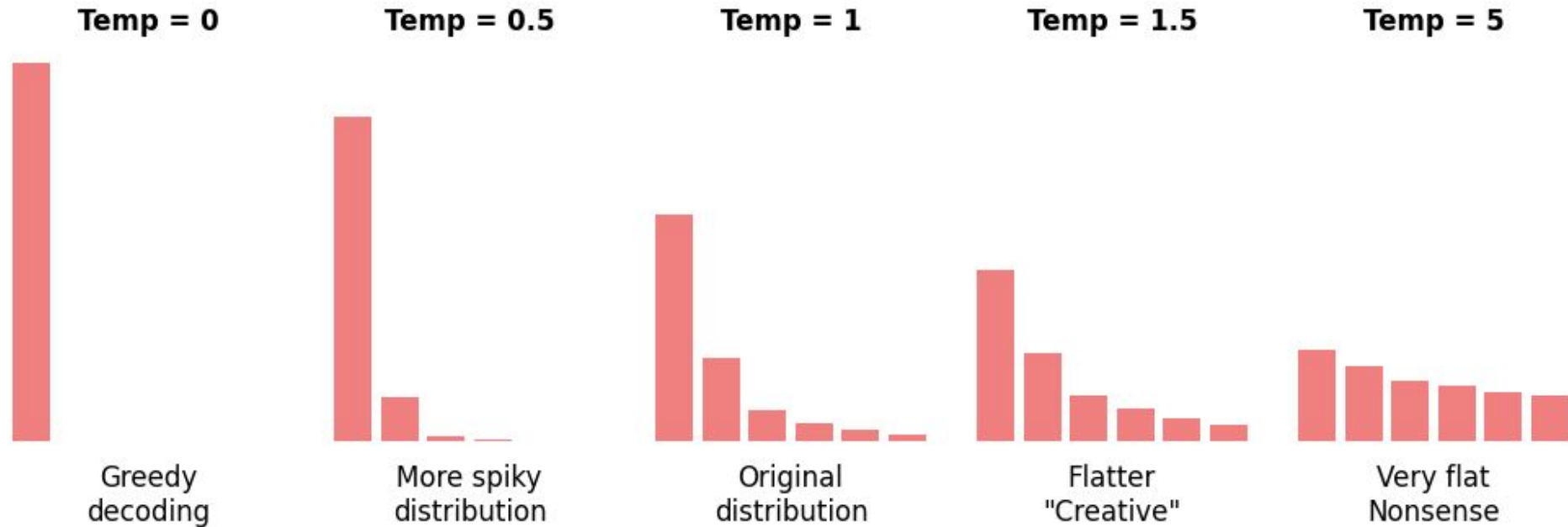
Instead of

$$\mathbf{y} = \text{softmax}(u)$$

We do

$$\mathbf{y} = \text{softmax}(u/\tau)$$

Effect of Temperature on Softmax Function



Top K Filtering

1. Choose # of words k
2. For each word in the vocabulary V , use the language model to compute the likelihood of this word given the context $p(w_t | w_{<t})$
3. Sort the words by likelihood, keep only the top k most probable words.
4. Renormalize the scores of the k words to be a legitimate probability distribution.
5. Randomly sample a word from within these remaining k most probable words according to its probability.

Top P Sampling

Problem with top-k: k is fixed so may cover very different amounts of probability mass in different situations

Idea: Instead, keep the top p percent of the probability mass.

Given a distribution $P(w|\mathbf{w}_{<t})$, the top-p vocabulary $V(p)$ is the smallest set of words such that

$$\sum_{w \in V(p)} P(w|\mathbf{w}_{<t}) \geq p$$

Visualization

<https://poloclub.github.io/transformer-explainer/>

Summary

- **Transformers unify NLP:** The Transformer architecture enables a single framework for representation, generation, and sequence-to-sequence tasks.
- **LLMs learn everything jointly:** Instead of separate components, modern models learn representations, context, and tasks end-to-end at scale.
- **Output is controllable:** At inference, techniques like top-k, top-p, and temperature let us trade off creativity, diversity, and reliability in generated text.